

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**



Disusun oleh :

Irfan Ardiansah Sulaeman      251511043

**PROGRAM STUDI D3 TEKNIK INFORMATIKA**  
**JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA**  
**POLITEKNIK NEGERI BANDUNG**

**2026**

## A. Circle dan Cylinder

### A.1 Persiapan (Step 1-3)

Source Code :

- *Circle.java*

```
- public class Circle {  
-     private double radius;  
-     private String color;  
-  
-     public Circle() {  
-         radius = 1.0;  
-         color = "red";  
-     }  
-  
-     public Circle(double radius) {  
-         this.radius = radius;  
-         color = "red";  
-     }  
-  
-     public double getRadius() {  
-         return radius;  
-     }  
-  
-     public double getArea() {  
-         return radius * radius * Math.PI;  
-     }  
-  
-     @Override  
-     public String toString () {  
-         return "Circle[radius =" + radius + ", color =" + color +  
- "]" ;  
-     }  
-  
- }
```

- *TestCircle.java*

```
- public class TestCircle {  
-     public static void main(String[] args) {  
-         Circle c1 = new Circle();  
-         Circle c2 = new Circle(2.5);  
-  
-         System.out.println("c1 radius = " + c1.getRadius());  
-         System.out.println("c1 area   = " + c1.getArea());  
-         System.out.println(c1);  
-         System.out.println();  
-     }  
- }
```

```

-         System.out.println("c2 radius = " + c2.getRadius());
-         System.out.println("c2 area   = " + c2.getArea());
-         System.out.println(c2);
-     }
- }

```

Output :

```

c1 area   = 3.141592653589793
Circle[radius =1.0, color =red]

c2 radius = 2.5
c2 area   = 19.634954084936208
Circle[radius =2.5, color =red]

```

Percobaan Step 2 :

Mengubah warna dan radius pada new Circle, setRadius dan setColor

```

public class TestCircle2 {
    public static void main(String[] args) {
        Circle c3 = new Circle(3.0, "blue");
        System.out.println("== Sebelum setter ==");
        System.out.println("radius = " + c3.getRadius());
        System.out.println("color   = " + c3.getColor());
        System.out.println("area    = " + c3.getArea());
        System.out.println(c3);

        c3.setRadius(4.5);
        c3.setColor("green");

        System.out.println();
        System.out.println("== Sesudah setter ==");
        System.out.println("radius = " + c3.getRadius());
        System.out.println("color   = " + c3.getColor());
        System.out.println("area    = " + c3.getArea());
        System.out.println(c3);
    }
}

```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-II
ptember 2026/Tes (Bukan untuk dikum
$ java TestCircle2
== Sebelum setter ==
radius = 3.0
color = blue
area = 28.274333882308138
Circle[radius =3.0, color =blue]

== Sesudah setter ==
radius = 4.5
color = green
area = 63.61725123519331
Circle[radius =4.5, color =green]
```

Jawaban Checkpoint 1-2 :

1. Tuliskan: mana state Circle? mana behavior Circle? apa perbedaan state dari dua object yang dibuat?

**Jawaban :**

- State: radius dan color (field private yang menyimpan data tiap object).
- Behavior: getRadius(), getArea(), toString(). Dua constructor juga berperan menentukan nilai awal state.
- Perbedaan dua object: c1 dibuat dengan new Circle() sehingga radius-nya 1.0. c2 dibuat dengan new Circle(2.5) sehingga radius-nya 2.5. Color keduanya "red", tapi tiap object punya salinan field sendiri, jadi area-nya juga beda (3.14... vs 19.63...).

2. Mengapa code di luar Circle tidak mengubah radius secara langsung? Apa peran method public terhadap state private?

**Jawaban :**

- Kenapa tidak bisa langsung: radius dideklarasikan private, jadi hanya bisa diakses dari dalam class Circle. Kalau TestCircle menulis c3.radius = 10, compiler menolaknya (error private access).
- Peran method public: getRadius(), setRadius(), getColor(), setColor() adalah jalur resmi untuk baca/ubah state. Class pemilik yang mengontrol caranya, misalnya nanti bisa ditambah validasi tanpa mengubah kode di luar.

Prediksi :

- a. Apakah radius dan color perlu ditulis ulang sebagai field Cylinder?
- b. Behavior apa dari Circle yang masih berguna untuk Cylinder?
- c. State apa yang hanya dimiliki Cylinder?

**Jawaban :**

- a. Tidak. Alas Cylinder adalah lingkaran, jadi radius dan color sudah dimiliki lewat Circle. Menulis ulang cuma bikin duplikasi.
- b. `getRadius()`, `setRadius()`, `getColor()`, `setColor()`, dan terutama `getArea()` karena luas alas dipakai untuk hitung volume (luas alas  $\times$  tinggi).
- c. Height

## A2. Cylinder extends Circle (Step 4-5)

*Cylinder.java*

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
}
```

```
}
```

*TestCylinder.java*

```
public class TestCylinder {  
    public static void main(String[] args) {  
        Cylinder c1 = new Cylinder();  
        Cylinder c2 = new Cylinder(10.0);  
        Cylinder c3 = new Cylinder(2.0, 10.0);  
  
        printCylinder("c1", c1);  
        printCylinder("c2", c2);  
        printCylinder("c3", c3);  
    }  
  
    private static void printCylinder(String name, Cylinder c) {  
        System.out.println(name);  
        System.out.println("radius = " + c.getRadius());  
        System.out.println("height = " + c.getHeight());  
        System.out.println("base area = " + c.getArea());  
        System.out.println("volume = " + c.getVolume());  
        System.out.println();  
    }  
}
```

Output :

```
linder 2020/Per temdan 4/STC  
$ java TestCylinder  
c1  
radius = 1.0  
height = 1.0  
base area = 3.141592653589793  
volume = 3.141592653589793  
  
c2  
radius = 1.0  
height = 10.0  
base area = 3.141592653589793  
volume = 31.41592653589793  
  
c3  
radius = 2.0  
height = 10.0  
base area = 12.566370614359172  
volume = 125.66370614359172
```

Jawaban Checkpoint 3 :

Karena Cylinder dideklarasikan dengan extends Circle:

```
public class Cylinder extends Circle {
```

c3.getRadius() dan c3.getArea() bisa dipanggil karena Cylinder extends Circle, sehingga method public Circle diwarisi oleh Cylinder. Method tersebut tidak perlu ditulis ulang di Cylinder.

### A.3 Bukti constructor chain (Step 6)

Output new Cylinder(2.0, 10.0) :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRF
mber 2026/Pertemuan 4/src
$ java TestChain
Circle(double) dipanggil
Cylinder(double,double) dipanggil
```

Output tanpa super(radius) :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRF
mber 2026/Pertemuan 4/src
$ java TestChain
Circle() dipanggil
Cylinder(double,double) dipanggil
```

Jawaban Checkpoint 4 :

Bagian Circle dikerjakan lebih dulu. Buktinya, output menampilkan "Circle(double) dipanggil" sebelum "Cylinder(double,double) dipanggil". Hal ini karena super(radius) memanggil constructor Circle terlebih dahulu untuk menginisialisasi radius dan color, baru kemudian constructor Cylinder mengisi height. Saat super(radius) dihapus, Java otomatis memanggil super(), sehingga yang dijalankan adalah Circle() dan radius kembali ke nilai default 1.0.

### A.4 Bukti akses private ditolak (Step 7)

Preediksi :

return radius; di Cylinder akan berhasil, karena Cylinder adalah turunan Circle jadi mestinya punya radius juga.

Tambahkan code pada *Cylinder.java* :

```
public double cobaAksesRadius() {  
    return radius;    // radius milik Circle dan private  
}
```

Hasil Compile :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Mat  
r 2026/Pertemuan 4/src  
$ javac *.java  
Cylinder.java:34: error: radius has private access in Circle  
    return radius;    // radius milik Circle dan private  
        ^  
1 error
```

Alasan error karena field radius dideklarasikan private di class Circle. Modifier private membatasi akses hanya untuk kode yang berada di dalam class Circle itu sendiri.

Perbaiki :

Code diubah menjadi return getRadius(); karena method tersebut public yang diwarisi circle sehingga encapsulation tetap terjaga.

```
public double cobaAksesRadius() {  
    return getRadius();  
}
```

Output setelah diperbaiki :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak A
2026/Test Step 7 (Ada yang kurang)/files
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 3.141592653589793
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 3.141592653589793
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 12.566370614359172
volume = 125.66370614359172
```

Jawaban Checkpoint 5 :

Extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi dapat diakses langsung dari subclass. Field tersebut tetap private milik superclass dan hanya bisa dijangkau lewat method public.

## A.5 Override `getArea()` dan bug `getVolume()` (Step 8)

Tambahkan Override pada *Cylinder.java* :

```
@Override
public double getArea() {
    return 2 * Math.PI * getRadius() * height
        + 2 * Math.PI * getRadius() * getRadius();
}
```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhian/Mingg
2026/Pertemuan 4/src
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 12.566370614359172

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 691.1503837897544

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 1507.9644737231006
```

getVolume() memanggil getArea() tanpa super. Karena object-nya Cylinder, Java menjalankan getArea() milik Cylinder (luas permukaan, 150.80), bukan milik Circle (luas alas, 12.57). Akibatnya volume menjadi 1507.96, padahal seharusnya 125.66.

## A.6 Perbaiki `super.getArea()` (Step 8)

Perbaiki :

```
public double getVolume() {
    return super.getArea() * height;
}
```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ar
2026/Pertemuan 4/src
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172
```

Karena `getVolume()` memanggil `getArea()` tanpa `super`, sehingga yang dijalankan adalah versi `Cylinder` (luas permukaan). Rumus volume membutuhkan luas alas, jadi hasilnya salah.

## A.7 Override `toString()` (Step 9)

Tambahkan `toString()` pada *Cylinder.java* :

```
@Override
public String toString() {
    return "Cylinder: subclass of " + super.toString()
        + ", height=" + height;
}
```

Tambahkan di *TestCylinder.java*, setelah `printCylinder("c3", c3)` ;

```
printCylinder("c1", c1);
    printCylinder("c2", c2);
    printCylinder("c3", c3);
    System.out.println(c3);
}
```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan
2026/Pertemuan 4/src
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172

Cylinder: subclass of Circle[radius =2.0, color =red], height=10.0
```

Output Cylinder: subclass of Circle[radius =2.0, color =red], height=10.0 muncul karena System.out.println(c3) otomatis memanggil method toString() milik object c3. Karena c3 adalah object Cylinder, Java menjalankan toString() yang di-override di Cylinder, bukan yang di Circle.

Ubah sementara nama method menjadi toStrng(), tetapi pertahankan @Override. :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan
2026/Pertemuan 4/src
$ javac *.java
Cylinder.java:42: error: method does not override or implement a method from a supertype
    @Override
    ^
1 error
```

hapus @Override :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan
2026/Pertemuan 4/src
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172

Circle[radius =2.0, color =red]
```

kembalikan ke code yang benar :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan/
2026/Pertemuan 4/src
$ java TestCylinder
c1
radius = 1.0
height = 1.0
base area = 12.566370614359172
volume = 3.141592653589793

c2
radius = 1.0
height = 10.0
base area = 69.11503837897544
volume = 31.41592653589793

c3
radius = 2.0
height = 10.0
base area = 150.79644737231007
volume = 125.66370614359172

Cylinder: subclass of Circle[radius =2.0, color =red], height=10.0
```

Tabel perbandingan:

| Kondisi                    | Hasil compile | Output println(c3)                                                 |
|----------------------------|---------------|--------------------------------------------------------------------|
| toString() + @Override     | Error         | tidak bisa dijalankan                                              |
| toString() tanpa @Override | Berhasil      | Circle[radius =2.0, color =red] (salah)                            |
| toString() + @Override     | Berhasil      | Cylinder: subclass of Circle[radius =2.0, color =red], height=10.0 |

Jawaban Checkpoint 7:

@Override tidak membuat inheritance terjadi. Fungsinya sebagai pengecekan dari compiler: memastikan method benar-benar menimpa method superclass. Kalau nama atau parameternya salah ketik, compiler langsung memberi error, sehingga kita tidak tanpa sadar membuat method baru yang tidak pernah dipakai.

## B. Shape hierarchy

### B.1 Superclass Shape (Step 10)

*Shape.java*

```
public class Shape {
    private String color;
    private boolean filled;

    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() { return color; }
    public void setColor(String color) { this.color = color; }
    public boolean isFilled() { return filled; }
    public void setFilled(boolean filled) { this.filled = filled; }

    @Override
    public String toString() {
        return "Shape[color=" + color + ", filled=" + filled + "]";
    }
}
```

*TestShape.java*

```
public class TestShape10 {
    public static void main(String[] args) {
        Shape s1 = new Shape();
        Shape s2 = new Shape("blue", false);

        System.out.println("s1 (default) : " + s1);
        System.out.println("s2 (parameter): " + s2);
    }
}
```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64
6/Pertemuan 4/src
$ java TestShape
s1 (default) : Shape[color=green, filled=true]
s2 (parameter): Shape[color=blue, filled=false]
```

Checkpoint 8 :

Pertanyaan : State apa yang cukup umum sehingga masuk akal dimiliki Circle maupun Rectangle?

Jawaban : color dan filled, karena setiap bentuk geometri pasti punya warna dan status terisi atau tidak, sehingga cukup ditulis sekali di Shape.

## B.2 Circle sebagai Specialization dari Shape (Step 11)

*Circle.java*

```
// Source code is decompiled from a .class file using FernFlower
decompiler (from IntelliJ IDEA).
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        this.height = (double)1.0F;
    }

    public Cylinder(double var1) {
        this.height = var1;
    }

    public Cylinder(double var1, double var3) {
        super(var1);
        this.height = var3;
    }

    public double getHeight() {
        return this.height;
    }

    public void setHeight(double var1) {
        this.height = var1;
    }

    public double getVolume() {
        return super.getArea() * this.height;
    }

    public double cobaAksesRadius() {
        return this.getRadius();
    }

    public double getArea() {
```

```

        return (Math.PI * 2D) * this.getRadius() * this.height + (Math.PI
* 2D) * this.getRadius() * this.getRadius();
    }

    public String toString() {
        String var10000 = super.toString();
        return "Cylinder: subclass of " + var10000 + ", height=" +
this.height;
    }
}

```

*TestCircle.java :*

```

public class TestCircle {
    public static void main(String[] args) {
        Circle c1 = new Circle();
        Circle c2 = new Circle(2.5);
        Circle c3 = new Circle(3.0, "blue", false);

        System.out.println("c1 area      = " + c1.getArea());
        System.out.println("c1 perimeter = " + c1.getPerimeter());
        System.out.println("c1 color     = " + c1.getColor());
        System.out.println("c1 filled    = " + c1.isFilled());
        System.out.println(c1);
        System.out.println();

        System.out.println("c2 area      = " + c2.getArea());
        System.out.println("c2 perimeter = " + c2.getPerimeter());
        System.out.println("c2 color     = " + c2.getColor());
        System.out.println("c2 filled    = " + c2.isFilled());
        System.out.println(c2);
        System.out.println();

        System.out.println("c3 area      = " + c3.getArea());
        System.out.println("c3 perimeter = " + c3.getPerimeter());
        System.out.println("c3 color     = " + c3.getColor());
        System.out.println("c3 filled    = " + c3.isFilled());
        System.out.println(c3);
    }
}

```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan  
6/Pertemuan 4/src  
• $ java TestCircle  
c1 area      = 3.141592653589793  
c1 perimeter = 6.283185307179586  
c1 color     = green  
c1 filled    = true  
Circle[radius=1.0, Shape[color=green, filled=true]]  
  
c2 area      = 19.634954084936208  
c2 perimeter = 15.707963267948966  
c2 color     = green  
c2 filled    = true  
Circle[radius=2.5, Shape[color=green, filled=true]]  
  
c3 area      = 28.274333882308138  
c3 perimeter = 18.84955592153876  
c3 color     = blue  
c3 filled    = false  
Circle[radius=3.0, Shape[color=blue, filled=false]]
```

Checkpoint 9 :

Pertanyaan : Pisahkan daftar berikut menjadi dua, behavior yang berasal dari Shape dan behavior yang spesifik milik Circle.

Jawaban :

| Berasal dari Shape                              | Spesifik milik Circle                               |
|-------------------------------------------------|-----------------------------------------------------|
| getColor(), setColor(), isFilled(), setFilled() | getRadius(), setRadius(), getArea(), getPerimeter() |

### B.3 Rectangle sebagai Specialization dari Shape (Step 12)

*Rectangle.java* :

```
public class Rectangle extends Shape {  
    private double width;  
    private double length;  
  
    public Rectangle() {  
        super();  
        width = 1.0;  
        length = 1.0;  
    }  
  
    public Rectangle(double width, double length) {  
        super();  
        this.width = width;  
        this.length = length;  
    }  
  
    public Rectangle(double width, double length, String color, boolean  
filled) {
```

```

        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }
    public double getLength() { return length; }
    public void setLength(double length) { this.length = length; }
    public double getArea() { return width * length; }
    public double getPerimeter() { return 2 * (width + length); }

    @Override
    public String toString() {
        return "Rectangle[width=" + width + ", length=" + length
            + ", " + super.toString() + "];"
    }
}

```

*TestRectangle.java :*

```

// Source code is decompiled from a .class file using FernFlower
decompiler (from IntelliJ IDEA).
public class TestRectangle12 {
    public TestRectangle12() {
    }

    public static void main(String[] var0) {
        printRect("r1 (default)", new Rectangle());
        printRect("r2 (width,length)", new Rectangle((double)3.0F,
(double)4.0F));
        printRect("r3 (lengkap)", new Rectangle((double)3.0F,
(double)4.0F, "yellow", true));
    }

    private static void printRect(String var0, Rectangle var1) {
        System.out.println(var0);
        System.out.println("area      = " + var1.getArea());
        System.out.println("perimeter = " + var1.getPerimeter());
        System.out.println(var1);
        System.out.println();
    }
}

```

Output :

```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhian/Minggu 4 30 September 202
6/Pertemuan 4/src
$ java TestRectangle
r1 (default)
area      = 1.0
perimeter = 4.0
Rectangle[width=1.0, length=1.0, Shape[color=green, filled=true]]

r2 (width,length)
area      = 12.0
perimeter = 14.0
Rectangle[width=3.0, length=4.0, Shape[color=green, filled=true]]

r3 (lengkap)
area      = 12.0
perimeter = 14.0
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]]
```

## Checkpoint 10

Pertanyaan : Apa kemiripan Circle dan Rectangle yang sudah ditarik ke Shape? Apa yang sengaja tetap berbeda?

Jawaban : Kemiripan yang ditarik ke Shape adalah atribut color dan filled beserta getter dan setter-nya. Yang sengaja tetap berbeda adalah atribut spesifik (radius pada Circle, width dan length pada Rectangle).

## B.4 Square sebagai Specialization dari Rectangle (Step 13-14)

*Square.java :*

```
public class Square extends Rectangle {
    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }
}
```

```

@Override
public String toString() {
    return "Square[side=" + getSide() + ", " + super.toString() +
"]";
}
}

```

4

*TestSquare.java :*

```

public class TestSquare {
    public static void main(String[] args) {
        Square s = new Square(5.0);
        System.out.println("awal : " + s);
        System.out.println("area : " + s.getArea());
        s.setWidth(8.0);
        System.out.println("sesudah setWidth(8): " + s);
    }
}

```

Output :

```

Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan/Minggu 4 30 S
6/Pertemuan 4/src
$ java TestSquare
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]
area : 25.0
sesudah setwidth(8): Square[side=8.0, Rectangle[width=8.0, length=5.0, Shape[color=green, filled=true]]

```

Tidak ada pesan error saat program dijalankan, tetapi hasilnya melanggar aturan persegi. Setelah `setWidth(8.0)` dipanggil, width berubah menjadi 8.0 sedangkan length tetap 5.0. Hal ini terjadi karena Square belum memiliki `setWidth()` sendiri sehingga yang dijalankan adalah versi Rectangle

Checkpoint 11 :

Pertanyaan : Inheritance dapat memberikan behavior yang secara teknis tersedia tetapi tidak selalu sesuai dengan aturan subclass. Di sinilah overriding dapat menjaga aturan object yang lebih spesifik.

Jawaban : Setter `setWidth()` yang diwarisi dari Rectangle secara teknis tersedia di Square, tetapi melanggar aturan persegi karena hanya mengubah satu sisi.

## B.5 Override setter agar Square tetap valid (Step 15)

*Square.java :*

```

public class Square extends Rectangle {
    public Square() {
        super(1.0, 1.0);
    }
}

```

```

    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setWidth(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setLength(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square[side=" + getSide() + ", " + super.toString() +
        "]\n";
    }
}

```

*TestSquare.java :*

```

public class TestSquare {
    public static void main(String[] args) {
        Square s = new Square(5.0);
        System.out.println("awal : " + s);
        System.out.println("area : " + s.getArea());
        s.setWidth(8.0);
        System.out.println("sesudah setWidth(8): " + s);
    }
}

```

```

        s.setLength(3.0);
        System.out.println("sesudah setLength(3): " + s);
        System.out.println("area sekarang : " + s.getArea());
        System.out.println("perimeter sekarang : " +
s.getPerimeter());
    }
}

```

Output :

```

Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhian/Minggu 4 30
6/Pertemuan 4/src
$ java TestSquare
awal : Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]]
area : 25.0
sesudah setWidth(8): Square[side=8.0, Rectangle[width=8.0, length=8.0, Shape[color=green, filled=true]]]
sesudah setLength(3): Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=green, filled=true]]]
area sekarang : 9.0
perimeter sekarang : 12.0

```

## Checkpoint 12

Pertanyaan : Jelaskan mengapa setWidth()/setLength() perlu dioverride, tetapi getArea()/getPerimeter() dapat tetap memakai implementasi Rectangle.

Jawaban :

setWidth() dan setLength() perlu dioverride karena versi Rectangle hanya mengubah satu sisi sehingga Square bisa kehilangan bentuk persegi. getArea() dan getPerimeter() hanya membaca width dan length, dan karena keduanya selalu sama setelah setter dijaga, rumus Rectangle ( $\text{width} \times \text{length}$  dan  $2 \times (\text{width} + \text{length})$ ) tetap menghasilkan nilai yang benar untuk Square.

## B.6 Test Hierarchy Shape (Step 16)

*TestShape.java*

```

public class TestShape {
    public static void main(String[] args) {
        Shape a = new Shape("black", false);
        Circle b = new Circle(2.0, "blue", true);
        Rectangle c = new Rectangle(3.0, 4.0, "yellow", true);
        Square d = new Square(5.0, "green", true);

        System.out.println(a);
        System.out.println(b + " area=" + b.getArea());
        System.out.println(c + " area=" + c.getArea());
        System.out.println(d + " area=" + d.getArea());
    }
}

```

Output :

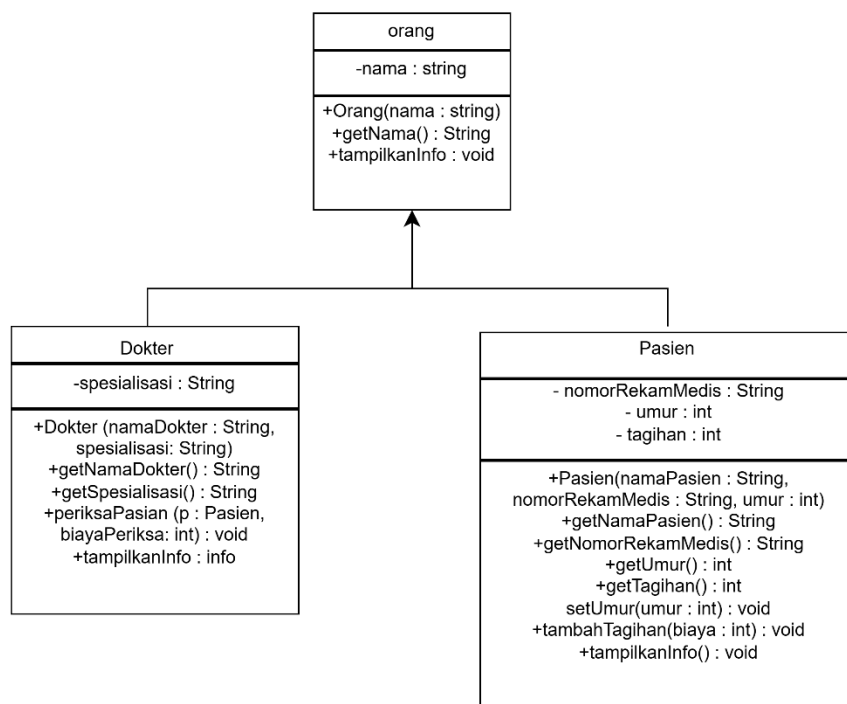
```
Irfan Ardiansah S@LAPTOP-LOQC0A7N-IRFAN MINGW64 /c/Kuliah/Matkul Semester 3/PBO/Praktikum Pak Ardhan/Minggu 4 30 Septem
6/Pertemuan 4/src
$ java TestShape
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0
```

## C. Kasus Tugas Sebelumnya

### C.1 Kandidat Generalization (Step 17)

|      | Dokter                                            | Pasien                                                        |
|------|---------------------------------------------------|---------------------------------------------------------------|
| sama | punya nama, getNama...(),<br>tampilkanInfo (nama) | punya nama, getNama...(),<br>tampilkanInfo (nama)             |
| beda | spesialisasi, periksaPasien()                     | nomorRekamMedis, umur, tagihan,<br>tambahTagihan(), setUmur() |

### C.2 Diagram Hierarchy (Step 18):



### C.3 Implementasi (Step 19)

Orang.java :

```
package id.ac.polban.model;

public class Orang {
    private String nama;
```

```

    public Orang(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    public void tampilkanInfo() {
        System.out.println("Nama : " + nama);
    }
}

```

*Pasien Java :*

```

package id.ac.polban.model;

public class Pasien extends Orang {
    private String nomorRekamMedis;
    private int umur;
    private int tagihan;

    public Pasien(String namaPasien, String nomorRekamMedis, int umur)
    {
        super(namaPasien);
        this.nomorRekamMedis = nomorRekamMedis;
        this.umur = umur;
        this.tagihan = 0;
    }

    public String getNamaPasien() {
        return getNama();
    }

    public String getNomorRekamMedis() {
        return nomorRekamMedis;
    }

    public int getUmur() {
        return umur;
    }

    public int getTagihan() {
        return tagihan;
    }

    public void setUmur(int umur) {

```

```

        if (umur > 0) {
            this.umur = umur;
        } else {
            System.out.println("Umur tidak valid \n");
        }
    }

    public void tambahTagihan(int biaya) {
        if (biaya > 0) {
            this.tagihan += biaya;
        }
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("No. RM : " + nomorRekamMedis);
        System.out.println("Umur : " + umur + " tahun");
        System.out.println("Total Tagihan : Rp" + tagihan);
        System.out.println("-----");
    }
}

```

*Dokter.java :*

```

package id.ac.polban.model;

public class Dokter extends Orang {
    private String spesialisasi;

    public Dokter(String namaDokter, String spesialisasi) {
        super(namaDokter);
        this.spesialisasi = spesialisasi;
    }

    public String getNamaDokter() {
        return getNama();
    }

    public String getSpesialisasi() {
        return spesialisasi;
    }

    public void periksaPasien(Pasien p, int biayaPeriksa) {
        System.out.println("Dr " + getNama() + "(" + spesialisasi + ")");
        memeriksa " + p.getNamaPasien());
        p.tambahTagihan(biayaPeriksa);
    }
}

```

```

@Override
public void tampilkanInfo() {
    super.tampilkanInfo();
    System.out.println("Spesialisasi : " + spesialisasi);
    System.out.println("-----");
}
}

```

*RumahSakit.java :*

```

import id.ac.polban.model.Dokter;
import id.ac.polban.model.Pasien;
import id.ac.polban.service.PemeriksaanService;

public class RumahSakit {
    public static void main(String[] args) {
        System.out.println("Selamat datang di Rumah Sakit\n");

        Pasien pasien1 = new Pasien("Zaki", "RM001", 21);
        Pasien pasien2 = new Pasien("Reza", "RM002", 20);

        Dokter dokter1 = new Dokter("Sahizidan", "Penyakit dalam");
        Dokter dokter2 = new Dokter("Wildan", "Anak");

        PemeriksaanService service = new PemeriksaanService();

        service.tambahDokter(dokter1);
        service.tambahDokter(dokter2);
        service.tambahPasien(pasien1);
        service.tambahPasien(pasien2);

        service.lakukanPemeriksaan(dokter1, pasien1, 150000);
        service.lakukanPemeriksaan(dokter2, pasien2, 120000);

        pasien1.setUmur(35);
        pasien2.setUmur(-2);

        System.out.println("- Data Pasien -");

        pasien1.tampilkanInfo();
        pasien2.tampilkanInfo();

        System.out.println("- Data Dokter -");

        dokter1.tampilkanInfo();
        dokter2.tampilkanInfo();
    }
}

```

```

        System.out.println("Total pemeriksaan yang dilakukan: " +
PemeriksaanService.getJumlahPemeriksaan());

    }
}

```

Output :

```

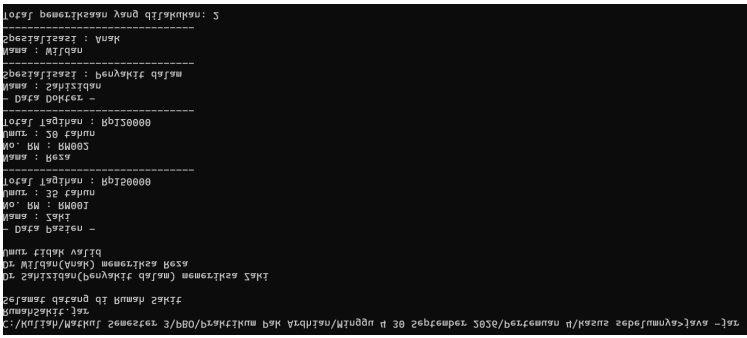
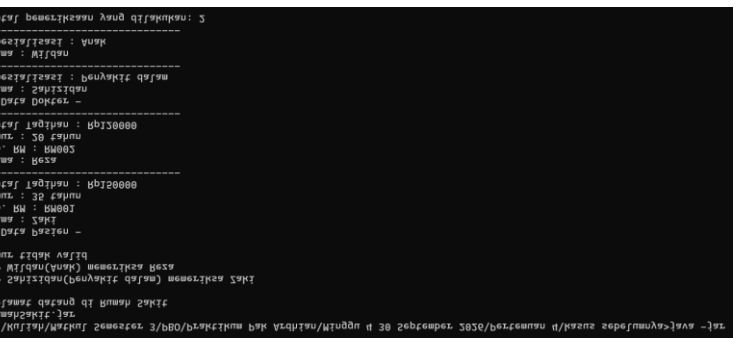
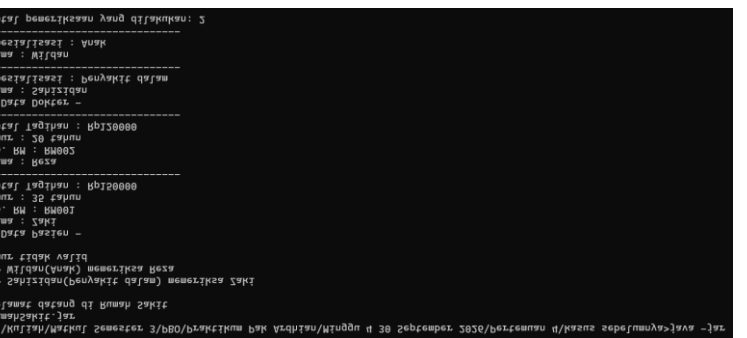
C:\Kuliah\Matkul Semester 3\PBO\Praktikum Pak Ardhian\Minggu 4 30 September 2026\Pertemuan 4\kasus sebelumnya>java -jar
RumahSakit.jar
Selamat datang di Rumah Sakit

Dr Sahizidan(Penyakit dalam) memeriksa Zaki
Dr Wildan(Anak) memeriksa Reza
Umur tidak valid

- Data Pasien -
Nama : Zaki
No. RM : RM001
Umur : 35 tahun
Total Tagihan : Rp150000
-----
Nama : Reza
No. RM : RM002
Umur : 20 tahun
Total Tagihan : Rp120000
-----
- Data Dokter -
Nama : Sahizidan
Spesialisasi : Penyakit dalam
-----
Nama : Wildan
Spesialisasi : Anak
-----
Total pemeriksaan yang dilakukan: 2

```

## TABEL SKENARIO UJI

| ID    | Skenario                                 | Status | Bukti                                                                                |
|-------|------------------------------------------|--------|--------------------------------------------------------------------------------------|
| T4-01 | Object subclass dibuat lewat constructor | PASS   |    |
| T4-02 | Method warisan dipanggil dari subclass   | PASS   |   |
| T4-03 | Method override dipanggil                | PASS   |  |
| T4-04 | Override memakai super.method(...)       | PASS   |  |

|       |                                  |      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|----------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T4-05 | Akses state sesuai encapsulation | PASS | <pre>public class Orang {     private String nama;      public String getNamaPasien() {         return getNama();     }      public String getNamaDokter() {         return getNama();     } }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| T4-06 | Dua subclass diuji               | PASS | Blok "Data Pasien" (No. RM, Umur, Tagihan) beda dengan "Data Dokter" (Spesialisasi)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| T4-07 | Regression tugas lama            | PASS | <p>Sebelum :</p> <pre>Selamat datang di Rumah Sakit  Dr Sahizidan(Penyakit dalam) memeriksa Zaki Dr Wildan(Anak) memeriksa Reza Umur tidak valid  - Data Pasien - Nama : Zaki No. RM : RM001 Umur : 35 tahun Total Tagihan : Rp150000 ----- Nama : Reza No. RM : RM002 Umur : 20 tahun Total Tagihan : Rp120000 ----- Total pemeriksaan yang dilakukan: 2</pre> <p>Sesudah :</p> <pre>Selamat datang di Rumah Sakit  Dr Sahizidan(Penyakit dalam) memeriksa Zaki Dr Wildan(Anak) memeriksa Reza Umur tidak valid  - Data Pasien - Nama : Zaki No. RM : RM001 Umur : 35 tahun Total Tagihan : Rp150000 ----- Nama : Reza No. RM : RM002 Umur : 20 tahun Total Tagihan : Rp120000 ----- - Data Dokter - Nama : Sahizidan Spesialisasi : Penyakit dalam ----- Nama : Wildan Spesialisasi : Anak ----- Total pemeriksaan yang dilakukan: 2</pre> |

## Self Assessment

| Pernyataan                                                             | Skor 1-5 |
|------------------------------------------------------------------------|----------|
| Saya dapat menjelaskan extends dari code yang saya jalankan.           | 3        |
| Saya dapat membedakan inherited method dan overridden method.          | 3        |
| Saya memahami mengapa constructor subclass memanggil super(...).       | 3        |
| Saya dapat menggunakan super.method(...) dengan alasan yang tepat.     | 2        |
| Saya dapat menjaga encapsulation pada hierarchy class.                 | 2        |
| Saya dapat menentukan kapan inheritance masuk akal pada kasus sendiri. | 2        |

## Refleksi - Jawab dari pengalaman praktikum

1. Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?  
Jawab :  
Karena radius, color, dan getArea() di Circle juga dibutuhkan Cylinder, sedangkan Cylinder hanya menambah height dan getVolume()
2. Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?  
Jawab :  
Object Cylinder bisa memakai method Circle tanpa menulis ulang, seperti c3.getRadius() dan c3.getArea()
3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?  
Jawab :  
Karena object Cylinder memuat bagian Circle yang harus diinisialisasi dulu lewat super(...). Di output, "Circle(double) dipanggil" muncul sebelum "Cylinder(double,double) dipanggil".
4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?  
Jawab :  
Karena getVolume() memanggil getArea() versi Cylinder (luas permukaan), bukan luas alas dari Circle.
5. Apa yang secara eksplisit Anda minta ketika menulis super.getArea() atau super.toString()?  
Jawab :  
Saya meminta Java menjalankan versi milik superclass (Circle), bukan versi override di subclass.
6. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?  
Jawab :  
Karena setter warisan hanya mengubah satu sisi, sehingga Square tidak lagi berbentuk persegi. Dengan override, width dan length selalu diubah bersamaan.
7. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.  
Jawab :  
Tidak. Pada kasus Rumah Sakit, "Dokter adalah sebuah Orang" masuk akal, jadi nama ditarik ke Orang. Tapi "Dokter adalah sebuah Pasien" tidak masuk akal, jadi keduanya tidak saling mewarisi walaupun mirip.

8. Lengkapi dengan bahasa Anda sendiri: "Inheritance bukan sekadar mengurangi duplikasi, tetapi ..."

Jawab :

menyatakan hubungan "adalah sebuah" antar class, dan subclass harus tetap menjaga aturan dirinya, seperti Square yang width-nya harus selalu sama dengan length

Link Github :

[Pertemuan 4 - inheritance](#)